

Comprehensive Technical Guide to Logistic Regression

Machine Learning Study & Reference Material

September 18, 2026

Abstract

This document provides an in-depth mathematical, statistical, and practical framework for **Logistic Regression**, a foundational supervised machine learning algorithm for binary and multiclass classification. Topics covered include mathematical derivation from log-odds to the sigmoid function, Maximum Likelihood Estimation (MLE), Binary Cross-Entropy loss, gradient descent optimization, Gauss-Markov-style classification assumptions, hyperparameter tuning, evaluation metrics, visual insights, and complete Python implementations using Scikit-Learn.

1 Problem Formulation & Output Characteristics

Logistic Regression is a supervised machine learning algorithm designed specifically for classification tasks. While Linear Regression models a continuous target variable $y \in \mathbb{R}$, Logistic Regression models the probability $P(y = 1 \mid \mathbf{x}) \in [0, 1]$ that a given input feature vector $\mathbf{x} = [x_1, x_2, \dots, x_m]^T$ belongs to a particular categorical class.

- **Problem Type:** Supervised Classification (Binary, Multinomial, and Ordinal).
- **Output Type:** Continuous probability value bounded between 0 and 1, subsequently converted into discrete class predictions via a decision threshold.
- **Core Variants:**
 1. **Binomial Logistic Regression:** Target variable has two discrete outcomes (e.g., 0 or 1, Pass or Fail, Benign or Malignant).
 2. **Multinomial Logistic Regression:** Target variable contains ≥ 3 unordered categories (e.g., Disease A vs. B vs. C). Uses the **Softmax function** in place of the Sigmoid function.
 3. **Ordinal Logistic Regression:** Target variable has ≥ 3 ordered categories with quantitative significance (e.g., Low, Medium, High).

2 Mathematical Foundation & Derivation

2.1 Linear Combination & The Sigmoid Function

The algorithm first computes a linear combination of input features $z \in \mathbb{R}$:

$$z = \mathbf{w} \cdot \mathbf{x} + b = \left(\sum_{j=1}^m w_j x_j \right) + b \quad (1)$$

where $\mathbf{w} = [w_1, w_2, \dots, w_m]^T$ represents feature weights (coefficients) and b denotes the bias term (intercept). To map the unbounded real value $z \in (-\infty, +\infty)$ into a valid probability $p \in (0, 1)$, Logistic Regression applies the **Sigmoid (Logistic) function** $\sigma(z)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z} \quad (2)$$

2.1.1 Key Mathematical Properties of the Sigmoid Function

- **Asymptotic Bounds:** $\lim_{z \rightarrow +\infty} \sigma(z) = 1$ and $\lim_{z \rightarrow -\infty} \sigma(z) = 0$.
- **Symmetry Point:** $\sigma(0) = 0.5$, serving as the default decision threshold.
- **Derivative:** $\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$. This simple formulation simplifies gradient computations during optimization.

2.2 Log-Odds, Odds Ratio, and Step-by-Step Derivation

The **Odds Ratio** defines the ratio of the probability of an event occurring to the probability of it not occurring:

$$\text{Odds} = \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} = e^z = e^{\mathbf{w} \cdot \mathbf{x} + b} \quad (3)$$

Taking the natural logarithm yields the **Logit (Log-Odds)** transformation, establishing a linear relationship between features and the log-odds of the dependent variable:

$$\ln \left[\frac{p(\mathbf{x})}{1 - p(\mathbf{x})} \right] = z = \mathbf{w} \cdot \mathbf{x} + b \quad (4)$$

Step-by-Step Algebraic Derivation from Log-Odds to Sigmoid Probability:

$$\begin{aligned} \ln \left[\frac{p(\mathbf{x})}{1 - p(\mathbf{x})} \right] &= \mathbf{w} \cdot \mathbf{x} + b \\ \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} &= e^{\mathbf{w} \cdot \mathbf{x} + b} \quad (\text{Exponentiating both sides}) \\ p(\mathbf{x}) &= e^{\mathbf{w} \cdot \mathbf{x} + b} (1 - p(\mathbf{x})) \\ p(\mathbf{x}) &= e^{\mathbf{w} \cdot \mathbf{x} + b} - e^{\mathbf{w} \cdot \mathbf{x} + b} \cdot p(\mathbf{x}) \\ p(\mathbf{x}) + e^{\mathbf{w} \cdot \mathbf{x} + b} \cdot p(\mathbf{x}) &= e^{\mathbf{w} \cdot \mathbf{x} + b} \\ p(\mathbf{x}) (1 + e^{\mathbf{w} \cdot \mathbf{x} + b}) &= e^{\mathbf{w} \cdot \mathbf{x} + b} \\ p(\mathbf{x}) &= \frac{e^{\mathbf{w} \cdot \mathbf{x} + b}}{1 + e^{\mathbf{w} \cdot \mathbf{x} + b}} = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} = \sigma(z) \end{aligned}$$

3 Objective Function & Maximum Likelihood Estimation (MLE)

3.1 Likelihood Function & Binary Cross-Entropy Loss

Unlike Linear Regression which uses Ordinary Least Squares (OLS), Logistic Regression estimates parameters $\mathbf{w}$ and b using **Maximum Likelihood Estimation (MLE)**. Assuming independent Bernoulli trials for each sample $i = 1, \dots, n$:

$$P(y_i | \mathbf{x}_i) = [p(\mathbf{x}_i)]^{y_i} [1 - p(\mathbf{x}_i)]^{1 - y_i} \quad \text{where } y_i \in \{0, 1\} \quad (5)$$

The total joint Likelihood Function across n observations is:

$$L(\mathbf{w}, b) = \prod_{i=1}^n [p(\mathbf{x}_i)]^{y_i} [1 - p(\mathbf{x}_i)]^{1 - y_i} \quad (6)$$

Taking the natural logarithm yields the **Log-Likelihood function** $\ell(\mathbf{w}, b)$:

$$\ell(\mathbf{w}, b) = \sum_{i=1}^n [y_i \ln p(\mathbf{x}_i) + (1 - y_i) \ln(1 - p(\mathbf{x}_i))] \quad (7)$$

In machine learning, we convert this maximization problem into a cost minimization problem by defining the **Binary Cross-Entropy (Log Loss)** cost function $J(\mathbf{w}, b) = -\frac{1}{n} \ell(\mathbf{w}, b)$:

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)] \quad (8)$$

Why Mean Squared Error (MSE) is Inappropriate for Logistic Regression

If Mean Squared Error $J(\mathbf{w}) = \frac{1}{n} \sum (\sigma(z_i) - y_i)^2$ were used with the non-linear sigmoid function, the resulting loss surface would be non-convex with numerous local minima, preventing Gradient Descent from reliably finding the global optimum. Log Loss guarantees a smooth, strictly **convex loss surface**.

3.2 Gradient Descent Optimization

To find parameters $\mathbf{w}$ and b that minimize $J(\mathbf{w}, b)$, we compute partial derivatives with respect to each weight w_j :

$$\frac{\partial J(\mathbf{w}, b)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (p(\mathbf{x}_i) - y_i) x_{ij} = \frac{1}{n} \sum_{i=1}^n (\sigma(\mathbf{w} \cdot \mathbf{x}_i + b) - y_i) x_{ij} \quad (9)$$

The parameter update step for learning rate $\alpha > 0$ is:

$$w_j \leftarrow w_j - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial w_j}, \quad b \leftarrow b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b} \quad (10)$$

4 Core Statistical Assumptions

For Logistic Regression to produce robust, un-biased, and statistically sound predictions, several key assumptions must be satisfied:

1. **Binary or Categorical Target:** The dependent variable must be discrete/categorical (0/1 for binomial, or extended via Softmax for multinomial).
2. **Independent Observations:** Data points must be mutually independent without temporal or spatial clustering (no autocorrelation).
3. **Linearity of Log-Odds:** Independent variables must have a linear relationship with the log-odds $\ln(p/(1-p))$, though they need not be linearly related to target y itself.
4. **Absence of Severe Multicollinearity:** Independent variables should not be highly correlated with one another, as high correlation inflates standard errors of coefficients.
5. **No Extreme Outliers:** Extreme outliers in feature space can disproportionately sway maximum likelihood estimates.
6. **Large Sample Size:** MLE relies on asymptotic properties; small sample sizes produce unstable parameter estimates and high variance.

5 Advantages & Limitations (Pros & Cons)

Table 1: Key Advantages and Limitations of Logistic Regression

Advantages (Pros)	Limitations (Cons)
High Interpretability: Exponentiated coefficients e^{w_j} directly provide Odds Ratios for feature changes.	Linear Decision Boundary: Assumes linear separability in log-odds space; fails on complex non-linear patterns without manual transformations.
Probabilistic Output: Provides calibrated probabilities rather than uncalibrated margin scores.	Sensitivity to Multicollinearity & Outliers: High correlation between features degrades coefficient stability.
Computational Efficiency: Fast training and inference with modest memory footprint; scalable to large datasets.	Requires Large Datasets: Maximum Likelihood Estimation requires adequate sample sizes to avoid overfitting.
Regularization Compatibility: Integrates L1/L2 penalties cleanly to handle high-dimensional spaces.	Over-simplification: Cannot model intricate feature interactions automatically compared to decision trees or neural networks.

6 Hyperparameter Tuning & Regularization

6.1 Regularization Formulations

To prevent overfitting on noise or collinear features, regularized loss functions modify $J(\mathbf{w})$:

- **L1 Regularization (Lasso):** $J_{L1}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_{j=1}^m |w_j|$. Drives non-essential feature weights strictly to zero, performing automatic feature selection.
- **L2 Regularization (Ridge):** $J_{L2}(\mathbf{w}) = J(\mathbf{w}) + \frac{\lambda}{2} \sum_{j=1}^m w_j^2$. Shrinks weight magnitudes uniformly toward zero, stabilizing multicollinearity.
- **Elastic Net:** Combines L1 and L2 penalties via mixing parameter l_1 -ratio.

6.2 Key Scikit-Learn Hyperparameters

- `C`: Inverse of regularization strength ($C = 1/\lambda$). Smaller values mean stronger regularization.
- `penalty`: Specifies regularization type ('l1', 'l2', 'elasticnet', 'none').
- `solver`: Optimization algorithm ('lbfgs', 'liblinear', 'sag', 'saga', 'newton-cg'). 'saga' supports L1/L2/ElasticNet on large datasets.
- `max_iter`: Maximum iteration count for solver convergence (default 100, often increased to 1000+).

7 Performance Evaluation Metrics

Table 2: Classification Performance Metrics for Logistic Regression

Metric	Mathematical Formula	Interpretation & Application
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Overall proportion of correct predictions. Misleading under class imbalance.
Precision	$\frac{TP}{TP+FP}$	Proportion of true positive class predictions out of all predicted positives.
Recall (Sensitivity)	$\frac{TP}{TP+FN}$	Proportion of actual positive instances correctly identified by the model.
F1-Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of Precision and Recall. Essential for imbalanced targets.
AUC-ROC	$\int_0^1 \text{TPR}(f) d(\text{FPR}(f))$	Area under ROC curve (TPR vs. $\text{FPR} = \frac{FP}{FP+TN}$). Evaluates ranking quality across thresholds.
AUC-PR	$\int_0^1 \text{Precision}(r) dr$	Area under Precision-Recall curve. Preferred metric for highly imbalanced datasets.

8 Visual Diagram & Structural Comparison

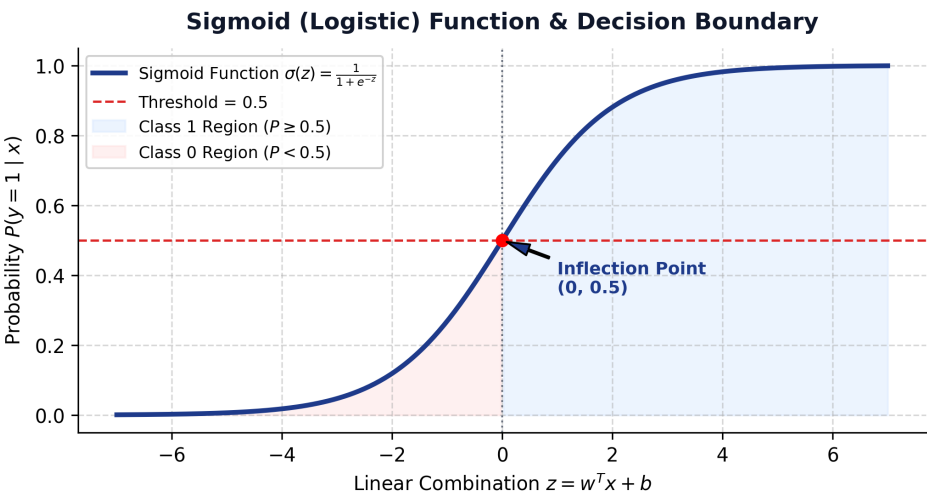


Figure 1: The Sigmoid (Logistic) Function curve mapping continuous linear values z to calibrated probabilities with a default decision threshold of 0.5.

8.1 Linear Regression vs. Logistic Regression Comparison Table

9 Executable Scikit-Learn Python Implementations

9.1 Binomial Logistic Regression (Breast Cancer Dataset)

Table 3: Structural Comparison: Linear Regression vs. Logistic Regression

Aspect	Linear Regression	Logistic Regression
Problem Type	Regression (Continuous target)	Classification (Discrete target)
Output Range	Continuous $(-\infty, +\infty)$	Bounded Probability $[0, 1] \rightarrow \{0, 1\}$
Hypothesis Curve	Straight best-fit line/hyperplane	S-shaped Sigmoid / Logistic Curve
Estimation Method	Ordinary Least Squares (OLS)	Maximum Likelihood Estimation (MLE)
Cost Function	Mean Squared Error (MSE)	Binary Cross-Entropy / Log Loss
Typical Metrics	MSE, RMSE, MAE, R^2 , Adj R^2	Accuracy, Precision, Recall, F1, AUC-ROC

```

from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, roc_auc_score

# 1. Load Dataset
X, y = load_breast_cancer(return_X_y=True)

# 2. Train-Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=23, stratify=y
)

# 3. Fit Logistic Regression Model
clf = LogisticRegression(max_iter=10000, random_state=0, C=1.0)
clf.fit(X_train, y_train)

# 4. Predict & Evaluate
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:, 1]

acc = accuracy_score(y_test, y_pred) * 100
auc = roc_auc_score(y_test, y_prob)

print(f"Model Accuracy: {acc:.2f}%")
print(f"AUC-ROC Score: {auc:.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

9.2 Multinomial Logistic Regression (Digits Dataset)

```

from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Load Multiclass Digits Dataset
digits = load_digits()
X, y = digits.data, digits.target

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# 3. Fit Multinomial Model using Softmax
multi_clf = LogisticRegression(
    multi_class='multinomial', solver='lbfgs', max_iter=10000, random_state=0
)
multi_clf.fit(X_train, y_train)

# 4. Evaluate Accuracy
y_pred = multi_clf.predict(X_test)
acc = accuracy_score(y_test, y_pred) * 100
print(f"Multinomial Logistic Regression Accuracy: {acc:.2f}%")

```